Hash tables

Searching in constant time

Mohammed Said BELAID
said.belaid@univ-usto.dz

14/10/2026



Outline

1 The search problem
2 From direct access to hashing
3 Collisions and chaining
4 Implementing chaining
5 Open addressing
6 Load factor
7 Any key: strings and objects
8 Hash tables in practice
9 Summary



Outline

1 The search problem
2 From direct access to hashing
3 Collisions and chaining
4 Implementing chaining
5 Open addressing
6 Load factor
7 Any key: strings and objects
8 Hash tables in practice
9 Summary



Finding a student

ID	Name
1023	Ahmed
1045	Mohammed
1087	Lina
1120	Sara

Question

How can we quickly find the name of the student with ID 1087?



Finding a student

ID	Name
1023	Ahmed
1045	Mohammed
1087	Lina
1120	Sara

Question

How can we quickly find the name of the student with ID 1087?
What if the list contains 100 000 students?



Naive solution: linear search

```
for (int i = 0; i < students.length; i++) {
    if (students[i].id == targetId) {
        return students[i].name;
    }
}
```

- Check entries one by one.
- In the worst case, inspect every entry.



Naive solution: linear search

```
for (int i = 0; i < students.length; i++) {
    if (students[i].id == targetId) {
        return students[i].name;
    }
}
```

- Check entries one by one.
- In the worst case, inspect every entry.
- Time complexity: $O(n)$.



Better solution: binary search

- If entries are sorted by ID, compare with the middle entry.
- Discard half of the remaining entries at each step.



Better solution: binary search

- If entries are sorted by ID, compare with the middle entry.
- Discard half of the remaining entries at each step.
- Time complexity: $O(\log n)$.



Better solution: binary search

- If entries are sorted by ID, compare with the middle entry.
- Discard half of the remaining entries at each step.
- Time complexity: $O(\log n)$.
- For $n = 10^6$: $\log_2 n \approx 20$ comparisons.



Can we do better?

- We want to look up a value using its key.



Can we do better?

- We want to look up a value using its key.
- We would like the lookup to be fast.



Can we do better?

- We want to look up a value using its key.
- We would like the lookup to be fast.
- Could we use the key to tell us where to look?



Outline

1 The search problem
2 From direct access to hashing
3 Collisions and chaining
4 Implementing chaining
5 Open addressing
6 Load factor
7 Any key: strings and objects
8 Hash tables in practice
9 Summary



Use the key as an index

```
String[] names = new String[2000];

names[1023] = "Ahmed";
names[1045] = "Mohammed";
names[1087] = "Lina";
names[1120] = "Sara";

// .......

System.out.println(names[1087]); // Lina
```

- No search is needed.
- Array access by index takes $O(1)$ time.



The problem with direct access

Example

Suppose IDs are:

- Big number: 8 700 087
- UUID: `5759456d-0ad2-4a3b-ba06-5bcac4f33481`
- Passport ID: `P<DZABENMOHAMED«MOHAMED 1234567891DZA8505204M2810243«<28`
- String: `"Name"`
- Any object type!

Problems

- A huge array may be required.
- Most positions may remain unused.
- Keys may not be integers.



The idea: calculate an index

Instead of using the key directly, calculate an array index from it.

Key	$\rightarrow$	Function	$\rightarrow$	Index
1087	$\rightarrow$	$f(1087)$	$\rightarrow$	7



The idea: calculate an index

Instead of using the key directly, calculate an array index from it.

Key	$\rightarrow$	Function	$\rightarrow$	Index
1087	$\rightarrow$	$f(1087)$	$\rightarrow$	7

Can we map many possible keys to a small array?



A first hash function

Suppose the array has 10 positions, indexed from 0 to 9: $h(k) = k \bmod 10$

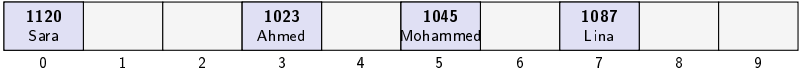
Key	Calculation	Index
1023	1023 mod 10	3
1045	1045 mod 10	5
1087	1087 mod 10	7
1120	1120 mod 10	0



A first hash function

Suppose the array has 10 positions, indexed from 0 to 9: $h(k) = k \bmod 10$

Key	Calculation	Index
1023	1023 mod 10	3
1045	1045 mod 10	5
1087	1087 mod 10	7
1120	1120 mod 10	0





What is a hash table?

A hash table stores key-value pairs in an array.

- A key identifies an entry.
- A value is the data associated with that key.
- A hash function helps determine where the entry should be stored.

$key \rightarrow hashfunction \rightarrow arrayindex \rightarrow value$



Properties of a useful hash function



Properties of a useful hash function

- Valid index: the final index must be within the table.
- Deterministic: the same key gives the same result.
- Efficient: calculating the result should be inexpensive.
- Good distribution: keys should spread reasonably across positions.



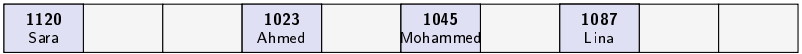
Outline

1 The search problem
2 From direct access to hashing
3 Collisions and chaining
4 Implementing chaining
5 Open addressing
6 Load factor
7 Any key: strings and objects
8 Hash tables in practice
9 Summary



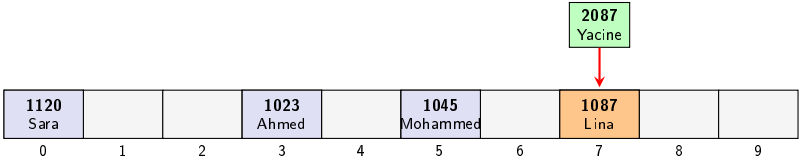
When two keys share an index

A new student arrives: Yacine, ID 2087.



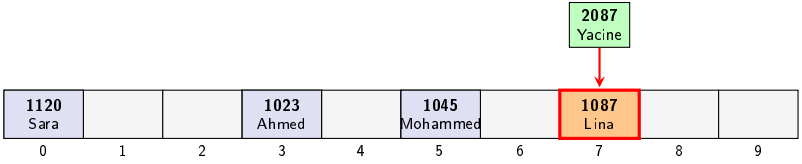
When two keys share an index

A new student arrives: Yacine, ID 2087. $h(2087) = 2087 \bmod 10 = 7$



When two keys share an index

A new student arrives: Yacine, ID 2087. $h(2087) = 2087 \bmod 10 = 7$



Slot 7 already holds Lina (1087): this is a collision.



Can we avoid every collision?

- Student IDs: thousands of possible keys.
- Our array: 10 positions.
- So several keys must share a position: 7, 1087, 2087, 3087, ... all map to slot 7.

A better hash function can reduce collisions, but cannot eliminate them in general.



Can we avoid every collision?

- Student IDs: thousands of possible keys.
- Our array: 10 positions.
- So several keys must share a position: 7, 1087, 2087, 3087, ... all map to slot 7.

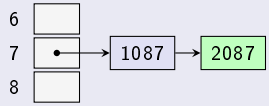
A better hash function can reduce collisions, but cannot eliminate them in general.
So where do we put Yacine?



Two main strategies

Separate chaining

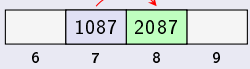
Each array position stores a list of entries.



2087 joins the list of slot 7.

Open addressing

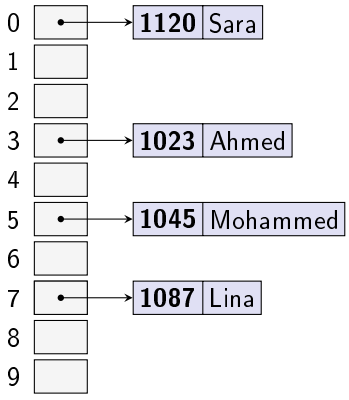
All entries stay inside the array. If a position is occupied, try another.



2087 takes the next free slot, 8.

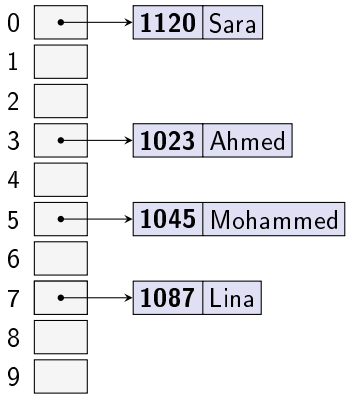


Inserting with chaining: an example



$m = 10$ buckets, $h(k) = k \bmod 10$

Inserting with chaining: an example

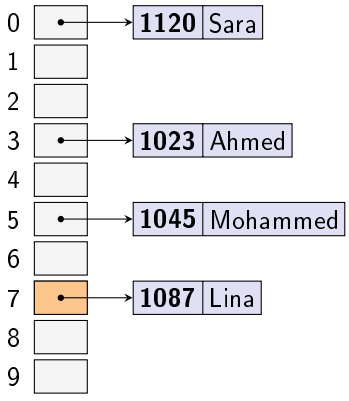


$m = 10$ buckets, $h(k) = k \bmod 10$

Insert $x = (2087, \text{Yacine})$



Inserting with chaining: an example

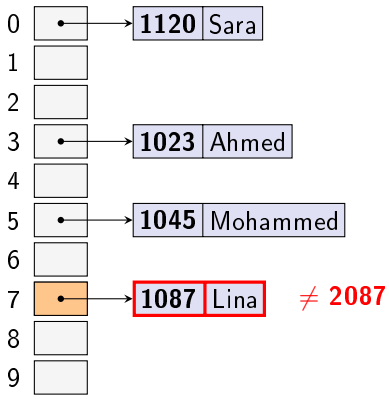


$m = 10$ buckets, $h(k) = k \bmod 10$

Insert $x = (2087, \text{Yacine})$

1. Compute the index:
   $h(2087) = 2087 \bmod 10 = 7$

Inserting with chaining: an example



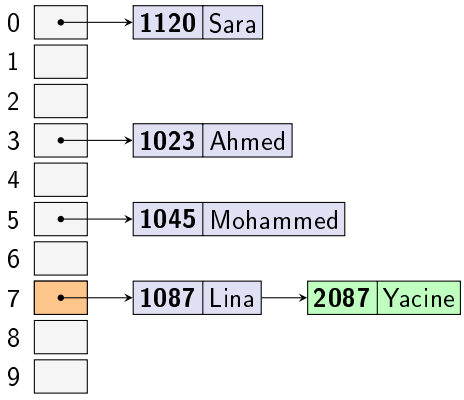
$m = 10$ buckets, $h(k) = k \bmod 10$

Insert $x = (2087, \text{Yacine})$

1. Compute the index:
   $h(2087) = 2087 \bmod 10 = 7$
2. Search 2087 in bucket 7:
   $2087 \neq 1087$, end of the chain



Inserting with chaining: an example



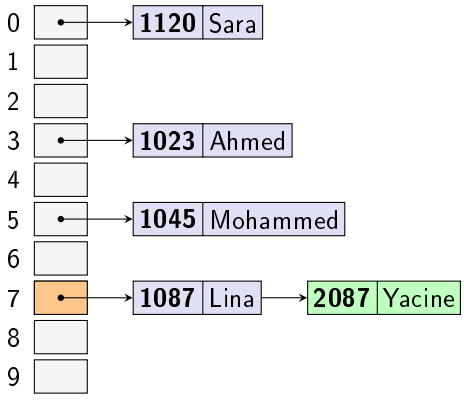
$m = 10$ buckets, $h(k) = k \bmod 10$

Insert $x = (2087, \text{Yacine})$

1. Compute the index:
   $h(2087) = 2087 \bmod 10 = 7$
2. Search 2087 in bucket 7:
   $2087 \neq 1087$, end of the chain
3. Key not found:
   append a new entry



Inserting with chaining: an example



$m = 10$ buckets, $h(k) = k \bmod 10$

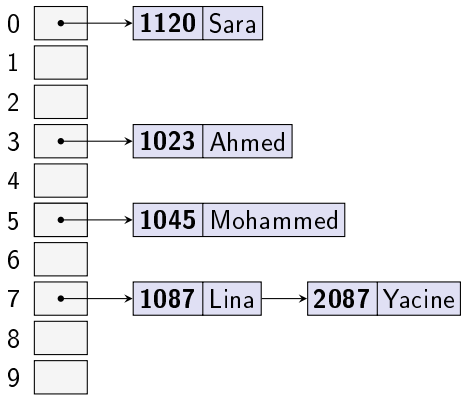
Insert $x = (2087, \text{Yacine})$

1. Compute the index:
   $h(2087) = 2087 \bmod 10 = 7$
2. Search 2087 in bucket 7:
   $2087 \neq 1087$, end of the chain
3. Key not found:
   append a new entry

Cost: $O(1)$ for the index
$+ O(\text{length of the chain})$

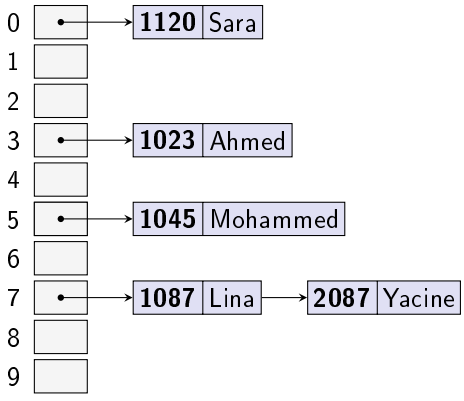


Inserting an existing key



$m = 10$ buckets, $h(k) = k \bmod 10$

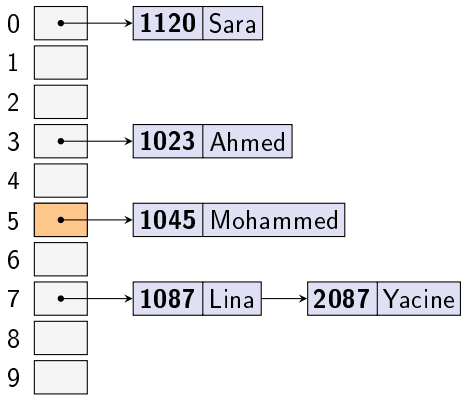
Inserting an existing key



$m = 10$ buckets, $h(k) = k \bmod 10$

Insert $x = (1045, \text{Mehdi})$

Inserting an existing key

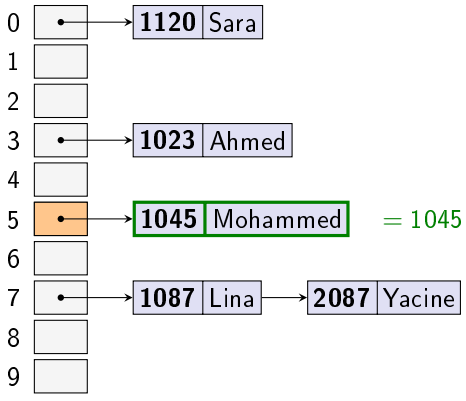


$m = 10$ buckets, $h(k) = k \bmod 10$

Insert $x = (1045, \text{Mehdi})$

1. Compute the index:
   $h(1045) = 1045 \bmod 10 = 5$

Inserting an existing key



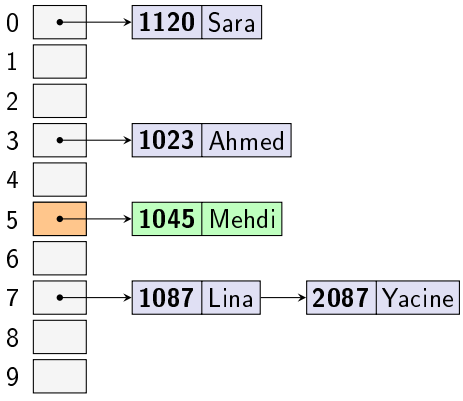
$m = 10$ buckets, $h(k) = k \bmod 10$

Insert $x = (1045, \text{Mehdi})$

1. Compute the index:
   $h(1045) = 1045 \bmod 10 = 5$
2. Search 1045 in bucket 5:
   $1045 = 1045$, found



Inserting an existing key



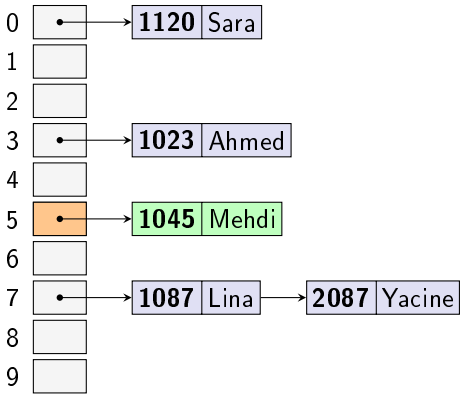
$m = 10$ buckets, $\quad h(k) = k \bmod 10$

Insert $x = (1045, \text{Mehdi})$

1. Compute the index:
   $h(1045) = 1045 \bmod 10 = 5$
2. Search 1045 in bucket 5:
   $1045 = 1045$, found
3. Replace the value:
   Mohammed $\rightarrow$ Mehdi



Inserting an existing key



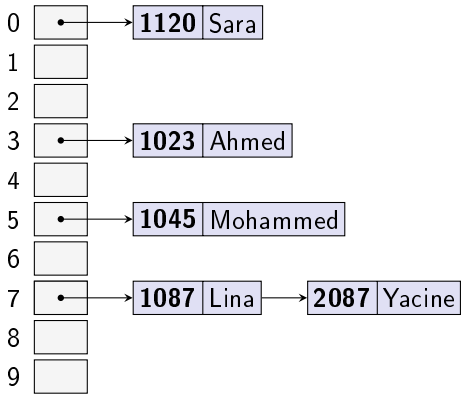
$m = 10$ buckets, $\quad h(k) = k \bmod 10$

Insert $x = (1045, \text{Mehdi})$

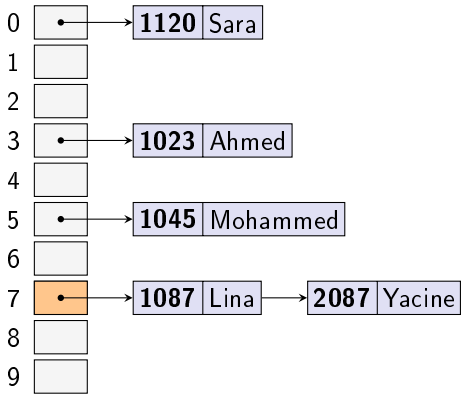
1. Compute the index:
   $h(1045) = 1045 \bmod 10 = 5$
2. Search 1045 in bucket 5:
   $1045 = 1045$, found
3. Replace the value:
   Mohammed $\rightarrow$ Mehdi

No new entry: a key appears at most once,
the size does not change.

Searching with chaining

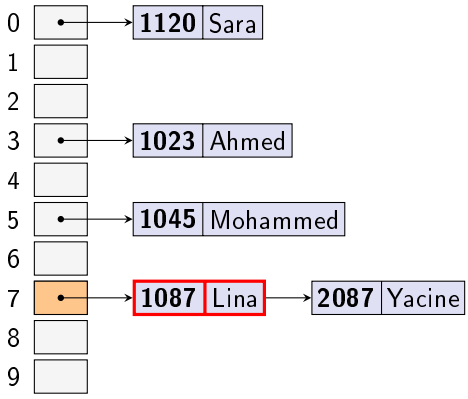


Searching with chaining



get(2087): $h(2087) = 7$

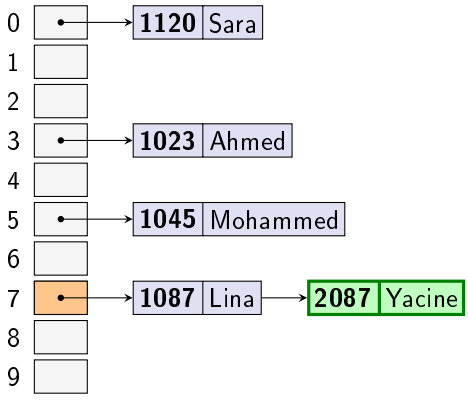
Searching with chaining



get(2087): $h(2087) = 7$

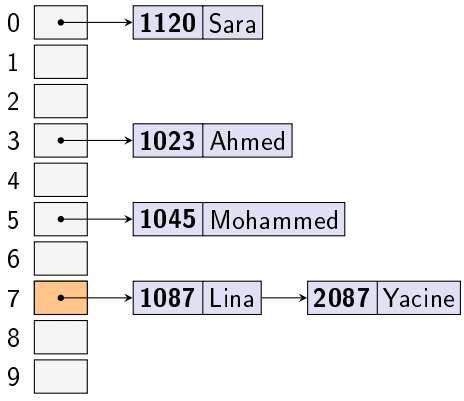
$1087 \neq 2087$: next entry

Searching with chaining



get(2087): $h(2087) = 7$
$1087 \neq 2087$: next entry
$2087 = 2087$: return Yacine

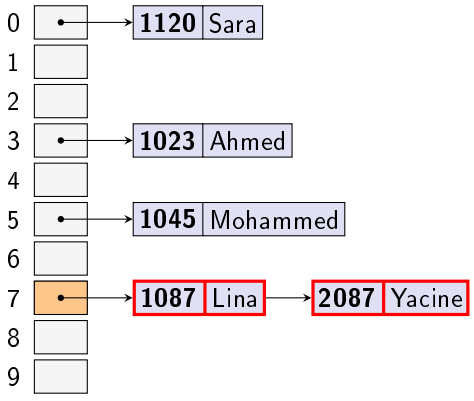
Searching with chaining



get(2087): $h(2087) = 7$
$1087 \neq 2087$: next entry
$2087 = 2087$: return Yacine

get(3087): $h(3087) = 7$

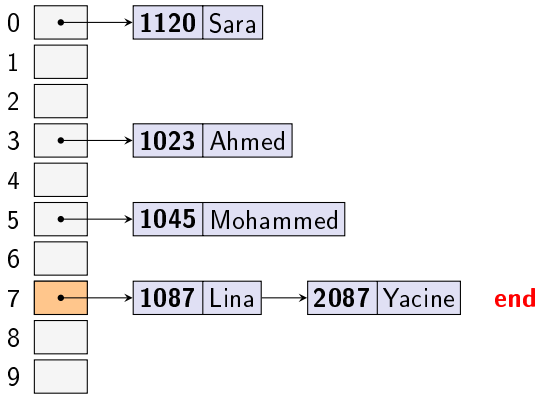
Searching with chaining



get(2087): $h(2087) = 7$
$1087 \neq 2087$: next entry
$2087 = 2087$: return Yacine

get(3087): $h(3087) = 7$
$1087 \neq 3087$, $2087 \neq 3087$

Searching with chaining

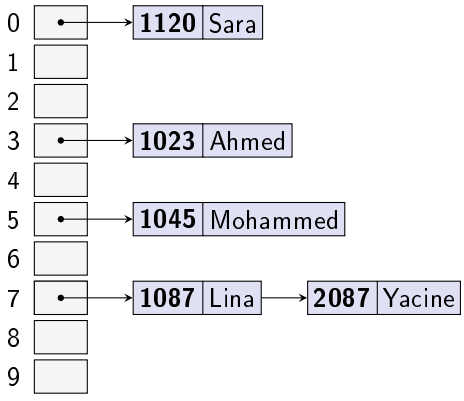


get(2087): $h(2087) = 7$
$1087 \neq 2087$: next entry
$2087 = 2087$: return Yacine

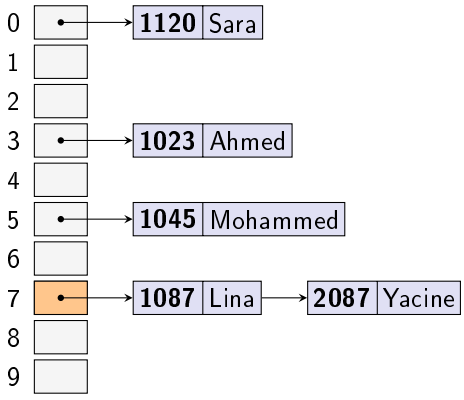
get(3087): $h(3087) = 7$
$1087 \neq 3087$, $2087 \neq 3087$
end of the chain: return `null`

Cost: one comparison per entry of the chain.

Deleting with chaining

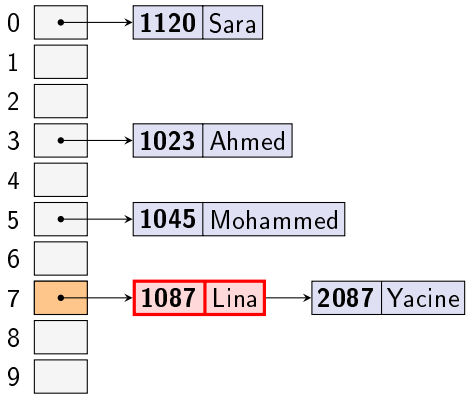


Deleting with chaining



`remove(1087)`: $h(1087) = 7$

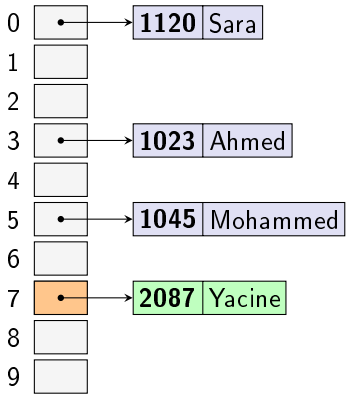
Deleting with chaining



`remove(1087)`: $h(1087) = 7$

1. Search 1087 in bucket 7: found

Deleting with chaining

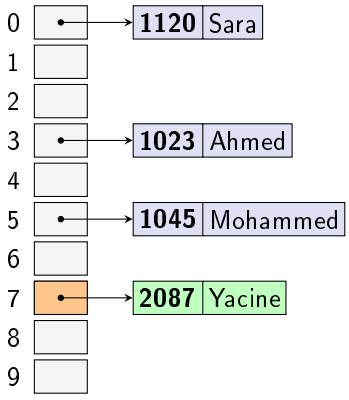


`remove(1087)`: $h(1087) = 7$

1. Search 1087 in bucket 7: found
2. Unlink the entry: bucket 7 now points to Yacine



Deleting with chaining



`remove(1087)`: $h(1087) = 7$

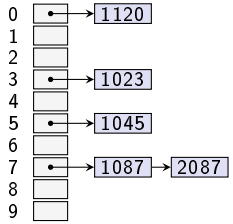
1. Search 1087 in bucket 7: found
2. Unlink the entry: bucket 7 now points to Yacine

The chain stays connected: a search for 2087 still works.



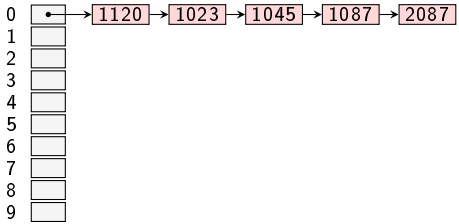
Cost of chaining

Good hash function



Short chains

Bad hash function, e.g. $h(k) = 0$



One long chain: a linked list!

- Average chain length: $\alpha = n/m$ (the *load factor*).
- Insert, search, delete: $O(1 + \alpha)$ on average.
- Worst case, all keys in one bucket: $O(n)$.



Outline

1 The search problem
2 From direct access to hashing
3 Collisions and chaining
4 Implementing chaining
5 Open addressing
6 Load factor
7 Any key: strings and objects
8 Hash tables in practice
9 Summary



Implementation plan

1. Structure: an array of buckets (one list per slot).
2. `hash`: from a key to an index.
3. `put`, `get`, `remove`: the animations, in code.
4. Test with our students.

Keys are `int` (student IDs), values are `String` (names).



A simple entry class

```
class Entry {
    int key;
    String value;

    Entry(int key, String value) {
        this.key = key;
        this.value = value;
    }
}
```



Implementing chaining: structure

```
class HashTable {
    private final LinkedList<Entry>[] buckets; // (1)

    HashTable(int capacity) {
        buckets = new LinkedList[capacity]; // (2)
        for (int i = 0; i < capacity; i++) {
            buckets[i] = new LinkedList<>(); // (4)
        }
    }
}
```

1. One linked list per bucket, as in the drawings.



Implementing chaining: structure

```
class HashTable {
    private final LinkedList<Entry>[] buckets; // (1)

    HashTable(int capacity) {
        buckets = new LinkedList[capacity]; // (2)
        for (int i = 0; i < capacity; i++) {
            buckets[i] = new LinkedList<>(); // (4)
        }
    }
}
```

1. One linked list per bucket, as in the drawings.
2. An array of linked lists.



Implementing chaining: structure

```
class HashTable {
    private final LinkedList<Entry>[] buckets; // (1)

    HashTable(int capacity) {
        buckets = new LinkedList[capacity]; // (2)
        for (int i = 0; i < capacity; i++) {
            buckets[i] = new LinkedList<>(); // (4)
        }
    }
}
```

1. One linked list per bucket, as in the drawings.
2. An array of linked lists.
3. Every bucket starts as an empty list.



Hash function for integer keys

```
int hash(int key) {
    return key % buckets.length;
}
```

- A method of `HashTable`: the capacity is `buckets.length`.
- Produces an index from 0 to `capacity-1` for keys ≥ 0.



Implementing chaining: put

```
void put(int key, String value) {
    int index = hash(key); // (1)
    for (Entry entry : buckets[index]) { // (2)
        if (entry.key == key) {
            entry.value = value; // (3)
            return;
        }
    }
    buckets[index].add(new Entry(key, value)); // (4)
}
```

1. Index of the bucket.



Implementing chaining: put

```
void put(int key, String value) {
    int index = hash(key); // (1)
    for (Entry entry : buckets[index]) { // (2)
        if (entry.key == key) {
            entry.value = value; // (3)
            return;
        }
    }
    buckets[index].add(new Entry(key, value)); // (4)
}
```

1. Index of the bucket.
2. Walk the chain of that bucket.



Implementing chaining: put

```
void put(int key, String value) {
    int index = hash(key); // (1)
    for (Entry entry : buckets[index]) { // (2)
        if (entry.key == key) {
            entry.value = value; // (3)
            return;
        }
    }
    buckets[index].add(new Entry(key, value)); // (4)
}
```

1. Index of the bucket.
2. Walk the chain of that bucket.
3. Key found: update the value.



Implementing chaining: put

```
void put(int key, String value) {
    int index = hash(key); // (1)
    for (Entry entry : buckets[index]) { // (2)
        if (entry.key == key) {
            entry.value = value; // (3)
            return;
        }
    }
    buckets[index].add(new Entry(key, value)); // (4)
}
```

1. Index of the bucket.
2. Walk the chain of that bucket.
3. Key found: update the value.
4. Key absent: append a new entry at the end of the list.



Implementing chaining: get

```
String get(int key) {
    int index = hash(key); // (1)
    for (Entry entry : buckets[index]) { // (2)
        if (entry.key == key) {
            return entry.value; // (3)
        }
    }
    return null; // (4)
}
```

1. Index of the bucket.



Implementing chaining: get

```
String get(int key) {
    int index = hash(key); // (1)
    for (Entry entry : buckets[index]) { // (2)
        if (entry.key == key) {
            return entry.value; // (3)
        }
    }
    return null; // (4)
}
```

1. Index of the bucket.
2. Walk the chain of that bucket.



Implementing chaining: get

```
String get(int key) {
    int index = hash(key); // (1)
    for (Entry entry : buckets[index]) { // (2)
        if (entry.key == key) {
            return entry.value; // (3)
        }
    }
    return null; // (4)
}
```

1. Index of the bucket.
2. Walk the chain of that bucket.
3. Key found: return its value.



Implementing chaining: get

```
String get(int key) {
    int index = hash(key); // (1)
    for (Entry entry : buckets[index]) { // (2)
        if (entry.key == key) {
            return entry.value; // (3)
        }
    }
    return null; // (4)
}
```

1. Index of the bucket.
2. Walk the chain of that bucket.
3. Key found: return its value.
4. End of the chain: the key is absent.



Implementing chaining: remove

```
boolean remove(int key) {
    int index = hash(key); // (1)
    Iterator<Entry> it = buckets[index].iterator(); // (2)
    while (it.hasNext()) {
        if (it.next().key == key) {
            it.remove(); // (3)
            return true;
        }
    }
    return false; // (4)
}
```

1. Index of the bucket.



Implementing chaining: remove

```
boolean remove(int key) {
    int index = hash(key); // (1)
    Iterator<Entry> it = buckets[index].iterator(); // (2)
    while (it.hasNext()) {
        if (it.next().key == key) {
            it.remove(); // (3)
            return true;
        }
    }
    return false; // (4)
}
```

1. Index of the bucket.
2. An iterator walks the chain and can remove the current entry.



Implementing chaining: remove

```
boolean remove(int key) {
    int index = hash(key); // (1)
    Iterator<Entry> it = buckets[index].iterator(); // (2)
    while (it.hasNext()) {
        if (it.next().key == key) {
            it.remove(); // (3)
            return true;
        }
    }
    return false; // (4)
}
```

1. Index of the bucket.
2. An iterator walks the chain and can remove the current entry.
3. Key found: unlink the entry.



Implementing chaining: remove

```
boolean remove(int key) {
    int index = hash(key); // (1)
    Iterator<Entry> it = buckets[index].iterator(); // (2)
    while (it.hasNext()) {
        if (it.next().key == key) {
            it.remove(); // (3)
            return true;
        }
    }
    return false; // (4)
}
```

1. Index of the bucket.
2. An iterator walks the chain and can remove the current entry.
3. Key found: unlink the entry.
4. Key absent: nothing changes.



Test with the students

```
HashTable students = new HashTable(10);

students.put(1120, "Sara");
students.put(1023, "Ahmed");
students.put(1045, "Mohammed");
students.put(1087, "Lina");
students.put(2087, "Yacine"); // collision with 1087
students.put(1045, "Mehdi"); // existing key: update

System.out.println(students.get(2087));
System.out.println(students.get(1045));
System.out.println(students.get(3087));
System.out.println(students.size());

students.remove(1087);
System.out.println(students.get(2087));
System.out.println(students.size());
```

Expected output:

```
Yacine
Mehdi
null
5
Yacine
4
```



Outline

1 The search problem
2 From direct access to hashing
3 Collisions and chaining
4 Implementing chaining
5 Open addressing
6 Load factor
7 Any key: strings and objects
8 Hash tables in practice
9 Summary



Strategy 2: open addressing

Instead of a list, use another slot of the array.
All entries are stored directly in the array.
If the preferred position is occupied, probe other positions until:

- an empty position is found, or
- the desired key is found.



Linear probing

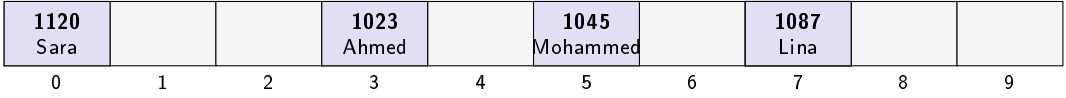
Try consecutive positions after a collision.

$$h_i(k) = (h(k) + i) \bmod m$$

- $h(k)$: initial index
- i: probe number, starting at 0
- m: table size



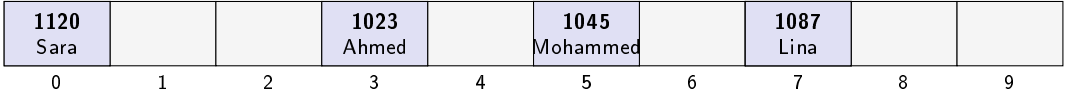
Linear probing: an example



$m = 10$ slots, $\quad h_i(k) = (k \bmod 10 + i) \bmod 10$



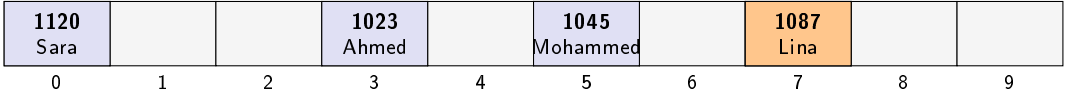
Linear probing: an example



$m = 10$ slots, $\quad h_i(k) = (k \bmod 10 + i) \bmod 10$

Insert $x = (2087, \text{Yacine})$

Linear probing: an example

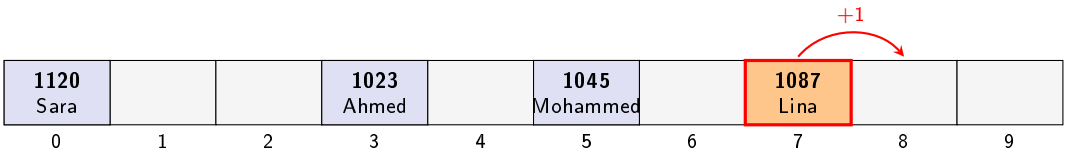


$m = 10$ slots, $\quad h_i(k) = (k \bmod 10 + i) \bmod 10$

Insert $x = (2087, \text{Yacine})$

1. Compute the index: $h_0(2087) = 2087 \bmod 10 = 7$

Linear probing: an example



$m = 10$ slots, $\quad h_i(k) = (k \bmod 10 + i) \bmod 10$

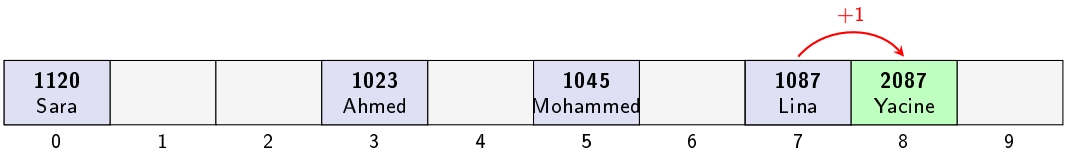
Insert $x = (2087, \text{Yacine})$

1. Compute the index: $h_0(2087) = 2087 \bmod 10 = 7$

2. Slot 7 holds $1087 \neq 2087$: probe the next slot, $h_1(2087) = 8$



Linear probing: an example



$m = 10$ slots, $h_i(k) = (k \bmod 10 + i) \bmod 10$

Insert $x = (2087, \text{Yacine})$

1. Compute the index: $h_0(2087) = 2087 \bmod 10 = 7$

2. Slot 7 holds $1087 \neq 2087$: probe the next slot, $h_1(2087) = 8$

3. Slot 8 is empty: store x there

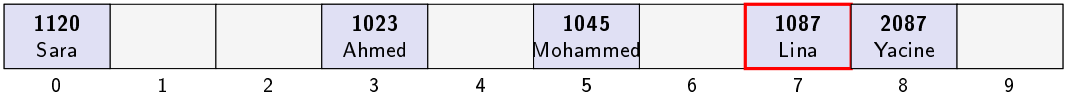


Searching with linear probing

0	1	2	3	4	5	6	7	8	9
1120 Sara			**1023** Ahmed		**1045** Mohammed		**1087** Lina	**2087** Yacine	



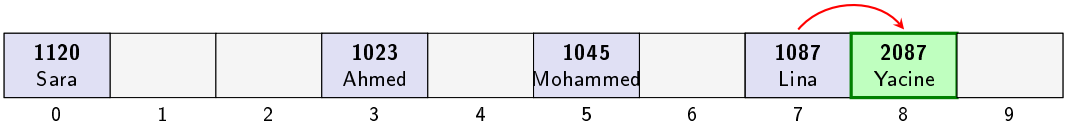
Searching with linear probing



get(2087): $h_0 = 7$: $1087 \neq 2087$

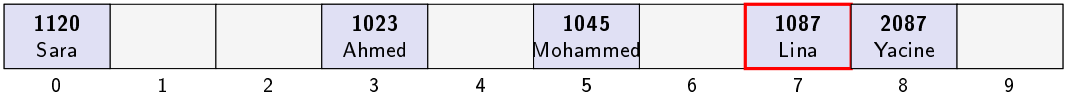


Searching with linear probing



get(2087): $h_0 = 7$: $1087 \neq 2087 \rightarrow h_1 = 8$: found, return Yacine

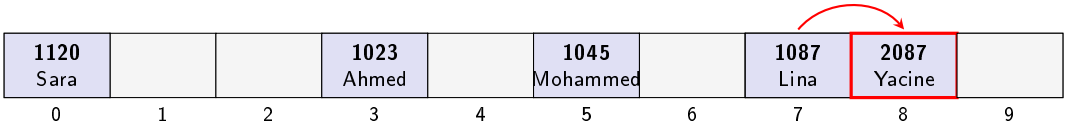
Searching with linear probing



get(2087): $h_0 = 7$: $1087 \neq 2087 \rightarrow h_1 = 8$: found, return Yacine

get(3087): $h_0 = 7$: $1087 \neq 3087$

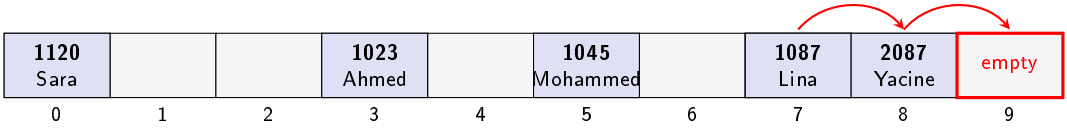
Searching with linear probing



get(2087): $h_0 = 7$: $1087 \neq 2087 \rightarrow h_1 = 8$: found, return Yacine

get(3087): $h_0 = 7$: $1087 \neq 3087 \rightarrow h_1 = 8$: $2087 \neq 3087$

Searching with linear probing



get(2087): $h_0 = 7$: $1087 \neq 2087 \rightarrow h_1 = 8$: found, return Yacine

get(3087): $h_0 = 7$: $1087 \neq 3087 \rightarrow h_1 = 8$: $2087 \neq 3087 \rightarrow h_2 = 9$: empty, return `null`

A search follows the same probe sequence as the insertion, and stops at the first empty slot.



Deleting with linear probing

0	1	2	3	4	5	6	7	8	9
1120 Sara			**1023** Ahmed		**1045** Mohammed		**1087** Lina	**2087** Yacine	



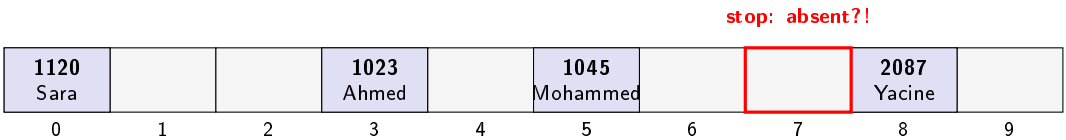
Deleting with linear probing

0	1	2	3	4	5	6	7	8	9
1120 Sara			**1023** Ahmed		**1045** Mohammed			**2087** Yacine	

`remove(1087)`, naive version: clear slot 7.



Deleting with linear probing

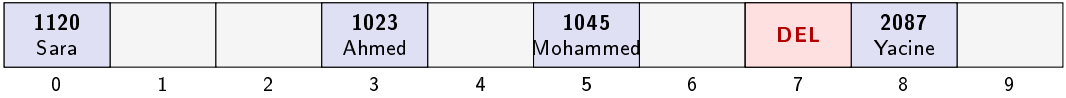


`remove(1087)`, naive version: clear slot 7.

`get(2087)`: slot 7 is empty, the search stops: 2087 is lost!

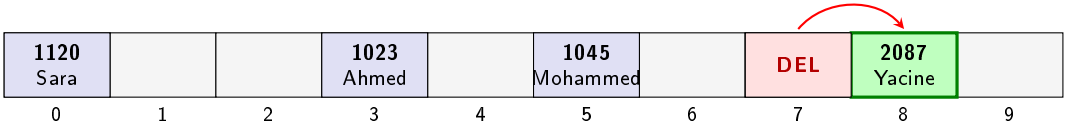


Deleting with linear probing



`remove(1087)`, naive version: clear slot 7.
`get(2087)`: slot 7 is empty, the search stops: 2087 is lost!
Fix: mark slot 7 as *deleted* (a *tombstone*).

Deleting with linear probing



`remove(1087)`, naive version: clear slot 7.
`get(2087)`: slot 7 is empty, the search stops: 2087 is lost!
Fix: mark slot 7 as *deleted* (a *tombstone*).
`get(2087)`: a tombstone does not stop the search → slot 8: found.



Other probing strategies

Linear probing fills neighbouring slots: entries form clusters. Other probe sequences jump further away:

- Quadratic probing: $h_i(k) = (h(k) + i^2) \bmod m$
  - for 2087 ($m = 10$): slots 7, 8, 1, 6, ...
- Double hashing: $h_i(k) = (h(k) + i \cdot h'(k)) \bmod m$
  - a second hash function h' chooses the step, so each key has its own sequence.



Other probing strategies

Linear probing fills neighbouring slots: entries form clusters. Other probe sequences jump further away:

- Quadratic probing: $h_i(k) = (h(k) + i^2) \bmod m$
  - for 2087 ($m = 10$): slots 7, 8, 1, 6, ...
- Double hashing: $h_i(k) = (h(k) + i \cdot h'(k)) \bmod m$
  - a second hash function h' chooses the step, so each key has its own sequence.

Same rules as linear probing: a search follows the same sequence, and deletion uses tombstones.



Chaining vs. linear probing

Property	Chaining	Linear probing
Storage	Bucket collections	Array only
Collisions	Add to same bucket	Search for another slot
More entries than slots	Yes, longer chains	No: the table can be full
Main concern	Long list	Clustering
Deletion	Remove from list	Tombstone



Exercise: linear probing

Implement a second version of the hash table, using open addressing with linear probing.

- Operations: `put(key, value)`, `get(key)`, `remove(key)`.
- Handle collisions using linear probing.
- Detect when the table is full.



Outline

1 The search problem
2 From direct access to hashing
3 Collisions and chaining
4 Implementing chaining
5 Open addressing
6 Load factor
7 Any key: strings and objects
8 Hash tables in practice
9 Summary



The load factor

How full is too full? We already met the measure in the cost of chaining:

$$\alpha = \frac{n}{m}$$

- n: number of stored entries
- m: number of table positions

Example: 7 entries in a table of capacity 10 gives $\alpha = 0.7$.



The load factor

How full is too full? We already met the measure in the cost of chaining:

$$\alpha = \frac{n}{m}$$

- n: number of stored entries
- m: number of table positions

Example: 7 entries in a table of capacity 10 gives $\alpha = 0.7$.
When α grows, chains (or probe sequences) get longer and operations slow down. With linear probing, α cannot exceed 1.
$\Rightarrow$ Performance depends on keeping α small.



Time complexity

With a good hash function and a bounded load factor:

Operation	Average case	Worst case
Search	$O(1)$	$O(n)$
Insertion	$O(1)$	$O(n)$
Deletion	$O(1)$	$O(n)$



Time complexity

With a good hash function and a bounded load factor:

Operation	Average case	Worst case
Search	$O(1)$	$O(n)$
Insertion	$O(1)$	$O(n)$
Deletion	$O(1)$	$O(n)$

So far, all our keys were `int`. What about other keys?



Outline

1 The search problem
2 From direct access to hashing
3 Collisions and chaining
4 Implementing chaining
5 Open addressing
6 Load factor
7 Any key: strings and objects
8 Hash tables in practice
9 Summary



What if the key is a string?

Remember the keys that broke direct access:

- UUID: `5759456d-0ad2-4a3b-ba06-5bcac4f33481`
- Passport ID, a name such as `"Sara"`, any object!

Our `HashTable` only accepts `int` keys.



What if the key is a string?

Remember the keys that broke direct access:

- UUID: `5759456d-0ad2-4a3b-ba06-5bcac4f33481`
- Passport ID, a name such as `"Sara"`, any object!

Our `HashTable` only accepts `int` keys. The idea, in two steps:

$$\text{key} \xrightarrow{\text{hash code}} \text{int} \xrightarrow{\text{mod } m} \text{index}$$

A string hash function combines information from its characters to produce an integer hash value.



Computing a string hash: Java's String.hashCode()

```
int hashCode(String s) {
    int h = 0;
    for (int i = 0; i < s.length(); i++) {
        h = 31 * h + s.charAt(i);
    }
    return h;
}
```

$$h(s) = s_0 \cdot 31^{n-1} + s_1 \cdot 31^{n-2} + \cdots + s_{n-1}$$

`hashCode("Lina")`:

Char	Code	$h = 31h + \text{code}$
L	76	76
i	105	2 461
n	110	76 401
a	97	2 368 528



Computing a string hash: Java's String.hashCode()

```
int hashCode(String s) {
    int h = 0;
    for (int i = 0; i < s.length(); i++) {
        h = 31 * h + s.charAt(i);
    }
    return h;
}
```

`hashCode("Lina")`:

Char	Code	$h = 31h + \text{code}$
L	76	76
i	105	2 461
n	110	76 401
a	97	2 368 528

$$h(s) = s_0 \cdot 31^{n-1} + s_1 \cdot 31^{n-2} + \cdots + s_{n-1}$$

- Every character counts, and so does its position: `"ab"` gives 3105, `"ba"` gives 3135.
- Long strings overflow `int`: the result can be negative.



Java: hashCode() and equals()

Java objects can provide:

- `hashCode()` to produce an integer hash code (step 1).
- `equals()` to determine whether two objects represent equal keys (it replaces `entry.key == key` in our chains).



From hash code to array index

A hash code is any `int`, possibly negative or larger than the table:

```
"Sara".hashCode()       //    2569501
"Mohammed".hashCode()   //  -443528454
```

The table converts it to a valid index in $[0, m-1]$:

```
int index = Math.abs(key.hashCode() % m);
```



Outline

1 The search problem
2 From direct access to hashing
3 Collisions and chaining
4 Implementing chaining
5 Open addressing
6 Load factor
7 Any key: strings and objects
8 Hash tables in practice
9 Summary



Where are hash tables useful?

- Find a student by ID.
- Count word frequencies.
- Detect duplicate values.
- Cache previously computed results.
- Track objects already processed.
- Index records by a key.



Our HashTable vs. Java HashMap

Java

```
Map<Integer, String> students = new HashMap<>();

students.put(1087, "Lina");
students.get(1087);     // Lina
students.remove(1087);
```

- Same operations, same idea: chaining, keys are unique.
- Plus: any key type, thanks to `hashCode()` and `equals()`.



Only keys: Java HashSet

What if we only need the keys, without values?

```
Set<String> names = new HashSet<>();

names.add("Ahmed");
names.add("Sara");
names.add("Ahmed");

System.out.println(names.size()); // 2
```

A HashSet stores unique elements and is useful for membership tests and duplicate detection.



Example: find the duplicate

```
boolean hasDuplicate(int[] a) {
    Set<Integer> seen = new HashSet<>();
    for (int x : a) {
        if (!seen.add(x)) {    // add() returns false if x is already there
            return true;
        }
    }
    return false;
}
```

Version	Time	Extra memory
Naive	$O(n^2)$	$O(1)$
Sort, then scan	$O(n \log n)$	$O(n)$ (copy)
Hash set	$O(n)$ on average	$O(n)$



Example: counting word frequencies

With values again: back to a `Map`.

```
Map<String, Integer> counts = new HashMap<>();

for (String word : words) {
    counts.put(word, counts.getOrDefault(word, 0) + 1);
}
```

The word is the key; its frequency is the value.



Choosing a map in Java

Type	Main characteristic
`HashMap`	General-purpose key-value lookup
`LinkedHashMap`	Predictable iteration order
`TreeMap`	Sorted keys (a search tree, not hashing)
`ConcurrentHashMap`	Concurrent access support



Hash tables in Python and C++

Python

```python
students = {}                    # dict
students[1087] = "Lina"
print(students[1087])            # Lina
print(students.get(3087))        # None
del students[1087]

names = {"Ahmed", "Sara", "Ahmed"}  # set
print(len(names))                # 2
```

C++

```cpp
std::unordered_map<int, std::string> students;
students[1087] = "Lina";
std::cout << students[1087];   // Lina
students.count(3087);          // 0
students.erase(1087);

std::unordered_set<std::string> names
    {"Ahmed", "Sara", "Ahmed"};
names.size();                  // 2
```

- Python's `dict` uses open addressing; C++'s `unordered_map` uses chaining.



Outline

1 The search problem
2 From direct access to hashing
3 Collisions and chaining
4 Implementing chaining
5 Open addressing
6 Load factor
7 Any key: strings and objects
8 Hash tables in practice
9 Summary



Key takeaways

- Direct access is the ideal case; hashing maps any key to an array position.
- Collisions are inevitable: chaining or open addressing resolves them.
- Lookup, insertion and deletion: $O(1)$ on average, $O(n)$ in the worst case.
- Hash quality and load factor matter.
- In Java, `equals()` and `hashCode()` must agree.
- Use standard collections in real applications; understand the implementation to use them well.



Open question: what if the table is full?

With linear probing, all entries live in the array: at most m entries ($\alpha \leq 1$).

- Our table has 10 slots and holds 10 students. Where does the 11th go?



Open question: what if the table is full?

With linear probing, all entries live in the array: at most m entries ($\alpha \leq 1$).

- Our table has 10 slots and holds 10 students. Where does the 11th go?
- Even before it is full: what happens to the probe sequences when $\alpha \to 1$?



Open question: what if the table is full?

With linear probing, all entries live in the array: at most m entries ($\alpha \leq 1$).

- Our table has 10 slots and holds 10 students. Where does the 11th go?
- Even before it is full: what happens to the probe sequences when $\alpha \to 1$?
- How could we fix it? What would the fix cost?

